

PayLock × Vendure Runtime Integration Evidence

Project Context

Environment:

- Vendure v3.6.3
- PostgreSQL 16 (Docker)
- Node.js runtime
- Custom PayLock plugin integrated into Vendure lifecycle

Project path:

```
C:\Projects\vendure-lab\paylock-vendure
```

Objective

Demonstrate that PayLock is not operating as an external utility, but as an embedded operational intelligence layer integrated directly into the Vendure commerce runtime.

The goal of this phase was to prove:

1. Runtime plugin loading
 2. EventBus lifecycle integration
 3. Order transition observation
 4. Persistent operational telemetry
 5. PostgreSQL persistence
 6. Real commerce runtime participation
-

Integration Architecture

Components

Vendure Runtime

- Shop API
- Admin API
- EventBus
- Order lifecycle

- Payment lifecycle

PayLock Plugin

Location:

```
src/plugins/paylock/
```

Core files:

```
paylock.plugin.ts  
paylock.entity.ts  
services/pay-lock.ts
```

Runtime Hooks

PayLock subscribes to:

```
OrderStateTransitionEvent  
PaymentStateTransitionEvent
```

using Vendure EventBus lifecycle integration.

Persistence Layer

Entity

```
PayLockSignal
```

Stored fields:

- orderCode
- fromState
- toState
- transitionType
- createdAt

Database Integration

PostgreSQL Runtime

Docker container:

```
paylock-vendure-postgres_db-1
```

Connection:

```
postgres:16-alpine
```

Database:

```
vendure
```

Persistence Verification

The following SQL query successfully returned persisted operational telemetry:

```
SELECT * FROM pay_lock_signal;
```

Verified Runtime Evidence

The following terminal output confirms successful runtime integration:

```
[PayLock][OrderTransition] Created -> AddingItems | Order Code: T1TM23BT2J38NCW4
```

Persistence confirmation:

```
[PayLock][DB] Saving signal...
```

Successful PostgreSQL persistence:

```
[PayLock][DB] Signal persisted successfully! ID: 1
```

Commerce Runtime Validation

A real Shop API mutation triggered the lifecycle transition:

```
mutation {  
  addItemToOrder(productVariantId: 1, quantity: 1) {  
    ... on Order {  
      id  
      code  
      state  
      totalWithTax  
    }  
  }  
}
```

Returned runtime result:

```
{  
  "data": {  
    "addItemToOrder": {  
      "id": "4",  
      "code": "T1TM23BT2J38NCW4",  
      "state": "AddingItems",  
      "totalWithTax": 2182820  
    }  
  }  
}
```

Key Engineering Outcome

PayLock successfully transitioned from:

```
Passive external logic
```

to:

Embedded commerce runtime intelligence layer

inside the Vendure operational lifecycle.

Current Capabilities

Operational Observation

- Order transition capture
- Payment transition capture
- Runtime event observation

Persistence

- PostgreSQL evidence storage
- Replayable telemetry
- Historical transition retention

Runtime Participation

- EventBus integration
 - Transaction-aware persistence
 - Lifecycle-aware execution
-

Next Planned Phases

Phase 1 — Decision Engine

- Confidence scoring
- Operational risk scoring
- Session continuity analysis
- Transition anomaly detection

Phase 2 — Advisory Enforcement

- Transition warnings
- Risk annotations
- Operational quarantine recommendations

Phase 3 — Sovereign Enforcement

- Deterministic progression gating
 - Hard transition blocking
 - Policy-driven commerce enforcement
-

GitHub Evidence Intent

This artifact is intended to serve as:

- Runtime integration proof
 - Commerce lifecycle evidence
 - Operational telemetry verification
 - Engineering milestone documentation
 - Future GitHub reference material
-

Status

Current status:

PayLock runtime integration with Vendure: SUCCESSFUL

Persistence layer:

ACTIVE

Operational telemetry:

CONFIRMED